КАК ЗАПУСКАТЬ ПРОЕКТ

Пошаговая инструкция для Windows-ПК с Visual Studio: запуск готового релиза, сборка из исходников, создание обычных отчетов, HTML-отчетов и импорт данных в Ghidra.

Дата подготовки: 02.07.2026

0. Что это за проект и какие отчеты он делает

EH/Unwind Analyzer анализирует Windows PE32+ AMD64 файлы: EXE, DLL, PYD и другие PE-файлы. Главная область проекта - exception/unwind metadata: .pdata, RUNTIME_FUNCTION и UNWIND_INFO. Статический режим просто читает файл как байты. Динамический режим запускает целевой EXE под Windows Debug API.

Консольный отчет	Печатается прямо в терминал. Его удобно сохранить в TXT через перенаправление > file.txt 2>&1.
JSON	Полный машинный отчет: метаданные, функции, эвристики, динамика, итоговая оценка. Открывается в VS Code, Notepad++, браузере или валидаторах.
HTML	Самодостаточный интерактивный отчет. Открывается двойным кликом или командой start. Интернет не нужен.
CSV	Таблица функций и unwind-данных. Удобно открывать в Excel или LibreOffice Calc.
Ghidra CSV	Специальный CSV для скрипта ghidra/import_unwind_csv.py. Нужен для разметки функций, комментариев и warning bookmarks в Ghidra.

Важно по безопасности: статический анализ не исполняет файл. Динамический режим с --dynamic исполняет целевой EXE. Для чужих или подозрительных файлов используйте отдельную Windows VM.

Что считается самым быстрым путем

Если нужно просто пользоваться проектом и смотреть отчеты, быстрее всего взять готовый архив release-output\eh-unwind-analyzer-1.0.0-windows-x64.zip, распаковать его и запускать bin\eh_unwind_analyzer.exe. Сборка через Visual Studio нужна, если преподаватель или заказчик хочет увидеть, что проект собирается из исходников.

1. Подготовка Windows-ПК

1.1. Что должно быть установлено

- Windows 10/11 x64.

- Visual Studio 2022 с workload Desktop development with C++.

- Компонент MSVC v143 x64/x86 build tools.

- Windows SDK, который ставится вместе с C++ workload.

- CMake. Он часто ставится вместе с Visual Studio, но можно поставить отдельно.

- Python 3, если нужно запускать тестовые Python-скрипты и валидаторы.

- Ghidra, если нужно импортировать Ghidra CSV и смотреть функции в CodeBrowser.

- Браузер: Edge, Chrome или Firefox для HTML-отчетов.

1.2. Как открыть правильный терминал Visual Studio

Для сборки MSVC удобнее всего использовать не обычный cmd, а x64 Native Tools Command Prompt for VS 2022. Он заранее настраивает PATH, INCLUDE, LIB и другие переменные так, чтобы cl.exe был доступен.

```
Пуск -> Visual Studio 2022 -> x64 Native Tools Command Prompt for VS 2022
```

В открывшемся окне проверьте инструменты:

```
cl
cmake --version
python --version
```

Нормально, если команда cl без аргументов выводит сведения о компиляторе и ошибку про отсутствие входных файлов. Плохо, если написано, что cl не найден. Тогда открыт не тот терминал или в Visual Studio не установлен C++ workload.

1.3. Как перейти в папку проекта

В командах ниже используется переменная PROJECT. Замените путь на тот, где реально лежит папка ip_full. Если путь содержит пробелы или русские буквы, обязательно оставляйте кавычки.

```
set PROJECT=C:\путь\к\ip_full
cd /d "%PROJECT%"
dir
```

Для текущей папки на этом компьютере пример выглядит так:

```
cd /d "C:\Users\Templar\Desktop\вкр 10 под minGW оконченный\ip_full"
```

> Совет: для динамического режима иногда проще сделать короткий ASCII-путь, например C:\EHUA\ip_full. Статический анализ обычно нормально работает и с русскими путями, но для отладки Windows меньше сюрпризов, когда путь короткий и без спецсимволов.

2. Способ A: запуск готового Windows-релиза без сборки

Этот способ нужен, если проект уже передан как готовый комплект и хочется сразу анализировать файлы. Компилятор здесь не нужен. Нужен только распакованный релизный архив.

2.1. Распаковать архив

```
cd /d "%PROJECT%"
cd release-output
powershell -NoProfile -Command "Expand-Archive -Force 'eh-unwind-analyzer-1.0.0-windows-x64.zip'
    'eh-unwind-analyzer-1.0.0-windows-x64'"
cd eh-unwind-analyzer-1.0.0-windows-x64
dir
```

После распаковки исполняемый файл находится здесь:

```
bin\eh_unwind_analyzer.exe
```

2.2. Проверить версию и справку

```
set TOOL=%CD%\bin\eh_unwind_analyzer.exe
"%TOOL%" --version
"%TOOL%" --help
```

В версии должно быть product 1.0.0 и report schema 1.1.0. В справке должны быть опции --json, --html, --csv, --ghidra-csv, --dynamic, --timeout-ms и --trace-steps.

2.3. Сделать отдельную папку для своих отчетов

```
mkdir my_reports
```

Лучше не сохранять новые отчеты прямо в bin или docs. Отдельная папка my_reports упрощает показ результата: в одном месте лежат TXT, JSON, HTML и CSV.

2.4. Запустить простой статический анализ

```
set TARGET=C:\путь\к\target.exe
"%TOOL%" "%TARGET%" --heuristics
```

Этот запуск ничего не записывает на диск. Он выводит результат в терминал: количество RUNTIME_FUNCTION, ошибки разбора, findings, score и verdict. Это удобно для быстрой проверки.

2.5. Сохранить обычный консольный отчет в TXT

```
"%TOOL%" "%TARGET%" --heuristics > "my_reports\console_report.txt" 2>&1
notepad "my_reports\console_report.txt"
```

Конструкция > file.txt 2>&1 сохраняет и обычный вывод, и ошибки в один файл. Такой TXT удобно отправлять вместе с HTML и JSON.

2.6. Сгенерировать полный набор отчетов: JSON, HTML, CSV

```
"%TOOL%" "%TARGET%" --heuristics ^
  --json "my_reports\report.json" ^
  --html "my_reports\report.html" ^
  --csv "my_reports\functions.csv"
```

Открытие результатов:

```
notepad "my_reports\report.json"
start "" "my_reports\report.html"
start "" "my_reports\functions.csv"
```

HTML-отчет самодостаточный: внутри уже есть стили и скрипты, интернет не нужен. CSV обычно открывается в Excel. JSON лучше смотреть в VS Code, Notepad++ или любом редакторе с подсветкой JSON.

2.7. Получить подробный dump RUNTIME_FUNCTION и unwind operations

```
"%TOOL%" "%TARGET%" --dump --limit 200 --ops > "my_reports\runtime_functions.txt" 2>&1
notepad "my_reports\runtime_functions.txt"
```

Опция --dump печатает записи RUNTIME_FUNCTION. --limit 200 ограничивает вывод первыми 200 записями, а --ops добавляет декодированные unwind operations. Это полезно, если нужно показать не только итоговый score, но и низкоуровневые данные.

3. Способ B: сборка из исходников через Visual Studio

3.1. Самый простой MSVC-вариант: build_msvc.bat

Откройте x64 Native Tools Command Prompt for VS 2022, перейдите в корень ip_full и запустите готовый bat-файл. Он вызывает cl.exe напрямую и кладет EXE в build\msvc-local.

```
cd /d "%PROJECT%"
build_msvc.bat
set TOOL=%CD%\build\msvc-local\eh_unwind_analyzer.exe
"%TOOL%" --version
```

Если появляется сообщение cl.exe not found, значит терминал открыт неправильно. Нужно именно x64 Native Tools Command Prompt, а не обычный cmd.

3.2. Полная сборка Visual Studio через CMake

Этот вариант лучше показывать как официальную сборку проекта: CMake создает Visual Studio build tree, затем собирается Release, после чего запускаются CTest-проверки.

```
cd /d "%PROJECT%"
cmake -S . -B build\msvc -G "Visual Studio 17 2022" -A x64 -DBUILD_TESTING=ON
cmake --build build\msvc --config Release --parallel
ctest --test-dir build\msvc -C Release --output-on-failure
set TOOL=%CD%\build\msvc\Release\eh_unwind_analyzer.exe
"%TOOL%" --version
```

Нормальный результат ctest - все тесты проходят. В релизном паспорте проекта зафиксировано 16/16 PASS для MSVC и 16/16 PASS для MinGW.

3.3. Сборка через CMake presets

В проекте есть CMakePresets.json. Пресеты используют генератор Ninja. Этот путь удобен, если Ninja установлен и доступен в PATH.

```
cd /d "%PROJECT%"
cmake --preset release
cmake --build --preset release
ctest --preset release
set TOOL=%CD%\build\release\eh_unwind_analyzer.exe
"%TOOL%" --version
```

3.4. MinGW-вариант, если нужен именно MinGW

```
cd /d "%PROJECT%"
build_mingw.bat
set TOOL=%CD%\build\mingw-local\eh_unwind_analyzer.exe
"%TOOL%" --version
```

Для build_mingw.bat нужен gcc из MinGW-w64 в PATH. Если gcc лежит не в PATH, можно задать переменную GCC полным путем к gcc.exe.

4. Практический маршрут: получить отчеты на встроенных примерах

В исходной папке есть тестовые PE-файлы. На них удобно проверять работу без поиска сторонних EXE. Ниже команды используют переменную TOOL, которую вы задали после сборки или после распаковки релиза.

4.1. Проверить benign_minimal.exe

```
cd /d "%PROJECT%"
mkdir my_reports
set TARGET=%CD%\tests\samples\bin\benign_minimal.exe
"%TOOL%" "%TARGET%" --heuristics ^
  --json "my_reports\benign_minimal.json" ^
  --html "my_reports\benign_minimal.html" ^
  --csv "my_reports\benign_minimal_functions.csv"
start "" "my_reports\benign_minimal.html"
```

Для benign_minimal ожидается спокойный результат: score 0, verdict no_strong_static_anomalies, evidence_level none. Это хороший первый демонстрационный запуск.

4.2. Проверить подозрительный пример

```
set TARGET=%CD%\tests\samples\bin\suspicious_eh_unwind_tamper.exe
"%TOOL%" "%TARGET%" --heuristics ^
  --json "my_reports\suspicious_tamper.json" ^
  --html "my_reports\suspicious_tamper.html" ^
  --csv "my_reports\suspicious_tamper_functions.csv"
start "" "my_reports\suspicious_tamper.html"
```

В HTML смотрите блоки Summary, Findings, Assessment и Functions. В JSON те же данные лежат в heuristic_analysis и assessment. CSV удобен для табличной сверки begin_rva, end_rva и статуса разбора unwind.

4.3. Сгенерировать Ghidra CSV для того же файла

```
"%TOOL%" "%TARGET%" --heuristics --ghidra-csv "my_reports\suspicious_tamper_ghidra.csv"
```

Этот CSV не заменяет обычный отчет. Он нужен именно для Ghidra-скрипта. В нем четыре ключевые колонки: begin_rva, end_rva, name, comment.

4.4. Запустить regression pack

```
python -m pip install -r requirements-dev.txt
python tests\run_regression_samples.py "%TOOL%"
```

Скрипт прогоняет встроенные образцы, проверяет JSON Schema и смысловые инварианты отчетов. Если все хорошо, по каждому sample будет [OK].

4.5. Где смотреть уже готовую evidence-базу проекта

reports\final_validation_2026-06-19\SUMMARY_RU.md	Итоговая сводка финальной проверки релиза 1.0.0.
reports\final_validation_2026-06-19\static_labeled	Отчеты по синтетическим статическим бинарникам с известными вмешательствами.
reports\final_validation_2026-06-19\static_real	Статическая проверка реального benign-корпуса.
reports\final_validation_2026-06-19\dynamic_smoke	Четыре контрольных динамических сценария VEH, VCH, UEF и runtime function table.
reports\final_validation_2026-06-19\dynamic_full	Полная динамическая evidence-база: JSON, CSV, REPORT.md и логи запусков.

> В финальной evidence-папке в основном лежат JSON, CSV, MD и LOG. HTML-отчеты создаются командой --html для выбранного PE-файла. Это нормально: HTML обычно генерируют под конкретный демонстрационный образец.

5. Динамический режим на Windows

Динамический режим включается опцией --dynamic. В этом режиме analyzer запускает target как дочерний процесс под отладчиком и собирает runtime evidence: исключения, VEH/VCH/UEF, обработчики, RtlAddFunctionTable, RtlInstallFunctionTableCallback, trace после handler hit.

> Не запускайте неизвестные файлы с --dynamic на основной системе. Для чужих EXE используйте изолированную Windows VM без важных данных. DLL/PYD напрямую динамически запускать нельзя: им нужен отдельный host process.

5.1. Обычный динамический запуск с HTML

```
set TARGET=C:\путь\к\target.exe
"%TOOL%" "%TARGET%" --heuristics --dynamic ^
  --timeout-ms 15000 ^
  --trace-steps 64 ^
  --json "my_reports\dynamic_report.json" ^
  --html "my_reports\dynamic_report.html"
start "" "my_reports\dynamic_report.html"
```

timeout-ms - жесткий лимит времени в миллисекундах. trace-steps - сколько single-step адресов собрать после попадания в handler breakpoint. Для обычной демонстрации 15000 и 64 достаточно.

5.2. Передать аргументы целевой программе

```
"%TOOL%" "%TARGET%" --heuristics --dynamic --json "my_reports\with_args.json" ^
  --html "my_reports\with_args.html" -- --input sample.dat --mode test
```

Все, что идет после отдельного разделителя --, передается уже target-программе, а не analyzer. Проект редактирует эти аргументы в report metadata, чтобы случайно не сохранить секреты вроде паролей.

5.3. Собрать и запустить dynamic smoke tests

Эти тесты нужны для демонстрации, что динамический collector видит VEH, UEF, VCH и runtime function table APIs. Сначала собираются специальные тестовые EXE в tests, затем запускается smoke pack.

```
cd /d "%PROJECT%"
cd tests
build_tests_msvc.bat
cd ..
tests\run_dynamic_smoke_windows.bat "%TOOL%"
```

Скрипт пишет tests\veh_dynamic_report.json, tests\uef_dynamic_report.json, tests\vch_dynamic_report.json, tests\rtl_dynamic_report.json и соответствующие *_dynamic_console.txt. В конце должно быть Dynamic smoke validation: OK.

5.4. Как читать dynamic-блок в JSON/HTML

- enabled=true означает, что пользователь включил --dynamic.

- ran=true означает, что динамическая сессия действительно стартовала.

- ok=true означает, что collector завершился без внутренней ошибки.

- timed_out=false означает, что target не был остановлен по timeout.

- target_exited=true означает, что целевой процесс завершился наблюдаемым образом.

- events показывает структурированную timeline: исключения, регистрации handlers, runtime table APIs и другие события.

- assessment.static_dynamic_correlation показывает, есть ли связь между статическими findings и runtime evidence.

6. Ghidra: как добавить данные EH/Unwind Analyzer

Ghidra-интеграция состоит из двух частей. Сначала analyzer создает специальный CSV через --ghidra-csv. Потом скрипт ghidra\import_unwind_csv.py импортирует CSV в открытую программу Ghidra.

6.1. Что делает Ghidra-скрипт

Создает функции	По begin_rva/end_rva скрипт создает функции в текущей программе Ghidra, если их еще нет.
Обновляет функции	Если функция уже есть, скрипт может обновить body и добавить комментарий.
Сохраняет ручные имена	Импорт осторожный: user/imported names не затираются без необходимости.
Добавляет комментарии	Перед функцией появляется pre-comment с префиксом [EH/Unwind Analyzer].
Ставит bookmarks	Если в комментарии есть parse failed или anomalies=, ставится Warning bookmark категории EH/Unwind Analyzer.
Пропускает конфликты	Если диапазон пересекается с чужой функцией или лежит вне памяти, строка CSV пропускается и печатается [skip].

6.2. Сгенерировать CSV для Ghidra

CSV обязательно должен быть создан для того же EXE, который потом открыт в Ghidra. Нельзя импортировать CSV от одного файла в другой файл: адреса и диапазоны не совпадут.

```
cd /d "%PROJECT%"
mkdir my_reports
set TARGET=%CD%\tests\samples\bin\suspicious_eh_unwind_tamper.exe
"%TOOL%" "%TARGET%" --heuristics ^
  --ghidra-csv "my_reports\suspicious_tamper_ghidra.csv" ^
  --html "my_reports\suspicious_tamper.html" ^
  --json "my_reports\suspicious_tamper.json"
```

Проверьте, что CSV создан:

```
type "my_reports\suspicious_tamper_ghidra.csv"
```

В первой строке должен быть заголовок begin_rva,end_rva,name,comment. Если файла нет, сначала смотрите ошибку analyzer в консоли.

6.3. Создать проект Ghidra

- Запустите Ghidra.

- Выберите File -> New Project.

- Выберите Non-Shared Project.

- Задайте имя, например EH_Unwind_Demo.

- Папку проекта Ghidra можно создать рядом с BKP, но не внутри build.

6.4. Импортировать EXE в Ghidra

- В главном окне Ghidra выберите File -> Import File.

- Выберите тот же TARGET, для которого был создан CSV.

- Убедитесь, что Ghidra определила формат как Portable Executable (PE).

- Нажмите OK.

- Дважды кликните по импортированному файлу, чтобы открыть CodeBrowser.

- Когда появится вопрос Analyze?, нажмите Yes или Analyze Now.

- Оставьте стандартные анализаторы включенными и нажмите Analyze.

Если Ghidra сама уже нашла часть функций, это нормально. Импортер не обязан создавать все заново: он может обновить существующие функции и добавить комментарии.

6.5. Подключить папку со скриптом import_unwind_csv.py

Скрипт лежит в папке проекта: ghidra\import_unwind_csv.py. Эту папку нужно добавить в Script Manager один раз.

- В CodeBrowser откройте Window -> Script Manager.

- Нажмите Manage Script Directories. В разных версиях Ghidra это кнопка со списком папок или маленькая иконка рядом с каталогами скриптов.

- Добавьте папку %PROJECT%\ghidra.

- Закройте окно управления директориями.

- В списке скриптов найдите import_unwind_csv.py. Обычно он находится в категории Analysis.

- Если скрипт не появился, нажмите Refresh в Script Manager или закройте и снова откройте Script Manager.

6.6. Запустить импорт CSV

- В Script Manager выделите import_unwind_csv.py.

- Нажмите Run Script.

- В появившемся окне выбора файла укажите my_reports\suspicious_tamper_ghidra.csv.

- Дождитесь завершения. Внизу в Console появится строка результата.

Пример нормального результата в консоли Ghidra:

```
EH/Unwind import complete: created=... updated=... renamed=... commented=... bookmarks=...
    skipped=0
```

created	Сколько новых функций создал скрипт.
updated	Сколько уже существующих функций обновил.
renamed	Сколько функций переименовал в импортированное имя, если это было безопасно.
commented	Сколько функций получили комментарий [EH/Unwind Analyzer].
bookmarks	Сколько warning bookmarks поставлено на parse failures или anomaly comments.
skipped	Сколько строк CSV пропущено из-за конфликта адресов, диапазонов или памяти.

6.7. Что смотреть в Ghidra после импорта

- Window -> Functions: список функций должен содержать импортированные/обновленные диапазоны.

- Listing: перед функциями должны появиться pre-comments с префиксом [EH/Unwind Analyzer].

- В комментариях ищите UNWIND v1/v2, flags, prolog, stack, frame, handler_rva и сообщения parse failed.

- Window -> Bookmarks: warning bookmarks категории EH/Unwind Analyzer показывают проблемные unwind-записи.

- Сравнивайте адреса так: Ghidra address = ImageBase + begin_rva. Для PE64 часто ImageBase равен 0x140000000.

Пример сверки RVA и адреса Ghidra:

```
begin_rva = 0x000013F0
ImageBase = 0x140000000
Ghidra address = 0x140000000 + 0x000013F0 = 0x1400013F0
```

6.8. Типовые проблемы Ghidra

Скрипт не виден	Не добавлена папка ip_full\ghidra в Script Manager. Добавьте ее через Manage Script Directories и нажмите Refresh.
CSV не подходит	CSV создан для другого EXE. Сгенерируйте --ghidra-csv заново для именно того файла, который открыт в Ghidra.
skipped > 0	Чаще всего диапазон пересекается с уже существующей функцией или лежит вне загруженной памяти. Проверьте, что EXE и CSV совпадают.
created=0	Это не ошибка, если Ghidra уже нашла функции. Смотрите updated и commented.
Нет bookmarks	Bookmarks появляются только для parse failed/anomalies. Для нормального файла bookmarks может быть 0.
Адреса выглядят другими	CSV хранит RVA, а Ghidra показывает VA. Прибавьте ImageBase.

6.9. Мини-чеклист для защиты по Ghidra

- Сначала показать HTML-отчет: где score, findings и assessment.

- Потом показать CSV для Ghidra: begin_rva, end_rva, name, comment.

- В Ghidra открыть тот же EXE.

- Запустить import_unwind_csv.py.

- Показать строку Console: commented > 0 и skipped=0.

- Открыть Listing и показать комментарий [EH/Unwind Analyzer].

- Открыть Bookmarks, если выбран файл с parse failed/anomalies.

- Объяснить, что Ghidra CSV - это визуальная разметка static evidence, а --dynamic - отдельное runtime evidence.

7. Частые ошибки и быстрые исправления

'cl' is not recognized	Открыт обычный cmd. Откройте x64 Native Tools Command Prompt for VS 2022 или установите C++ workload.
cmake не найден	Установите CMake или добавьте CMake из Visual Studio в PATH.
ninja не найден	Для CMake presets нужен Ninja. Используйте Visual Studio generator или установите Ninja.
tool not found	Неверно задан TOOL. Проверьте путь через dir "%TOOL%" или выполните "%TOOL%" --version.
Failed to read file	Неверный TARGET, нет доступа или путь без кавычек при пробелах.
Unknown/invalid option	Опечатка в ключе. Сверьтесь с "%TOOL%" --help.
Invalid value for --timeout-ms	timeout-ms должен быть больше 0.
HTML не открывается	Проверьте, что файл создан. Используйте start "" "path\report.html".
JSON пустой или старый	Проверьте return code analyzer и stderr. Writer использует atomic replacement, поэтому при ошибке старый отчет может сохраниться.
Dynamic mode зависает	Уменьшите timeout-ms, используйте безопасный тестовый EXE, запускайте в VM.
Антивирус блокирует EXE	Для своих тестовых бинарников добавьте исключение или запускайте в изолированной VM. Не отключайте защиту для неизвестных файлов на основной системе.

7.1. Как быстро проверить, что все готово к демонстрации

```
cd /d "%PROJECT%"
"%TOOL%" --version
"%TOOL%" "%CD%\tests\samples\bin\benign_minimal.exe" --heuristics ^
  --json "my_reports\demo.json" ^
  --html "my_reports\demo.html" ^
  --csv "my_reports\demo.csv" ^
  --ghidra-csv "my_reports\demo_ghidra.csv"
start "" "my_reports\demo.html"
```

Если эта последовательность прошла, то базовая демонстрация готова: EXE запускается, HTML открывается, JSON/CSV созданы, Ghidra CSV готов для импорта.

8. Короткий сценарий показа проекта преподавателю или заказчику

1. Показать структуру папки

```
dir
```

2. Показать версию analyzer

```
"%TOOL%" --version
```

3. Собрать проект из исходников

```
build_msvc.bat
```

4. Запустить быстрый benign sample

```
"%TOOL%" "%CD%\tests\samples\bin\benign_minimal.exe" --heuristics
```

5. Создать HTML/JSON/CSV

```
"%TOOL%" "%CD%\tests\samples\bin\suspicious_eh_unwind_tamper.exe" --heuristics --json
    "my_reports\s.json" --html "my_reports\s.html" --csv "my_reports\s.csv"
```

6. Открыть HTML

```
start "" "my_reports\s.html"
```

7. Создать Ghidra CSV

```
"%TOOL%" "%CD%\tests\samples\bin\suspicious_eh_unwind_tamper.exe" --heuristics --ghidra-csv
    "my_reports\s_ghidra.csv"
```

8. В Ghidra импортировать EXE

```
File -> Import File -> выбрать тот же EXE -> Analyze Now
```

9. В Ghidra запустить скрипт

```
Window -> Script Manager -> import_unwind_csv.py -> Run Script -> выбрать s_ghidra.csv
```

10. Показать evidence-базу

```
reports\final_validation_2026-06-19\SUMMARY_RU.md
```

> Главная мысль для объяснения: проект не просто печатает score. Он строит проверяемый pipeline: PE parser -> RUNTIME_FUNCTION -> UNWIND_INFO -> heuristics -> JSON/HTML/CSV -> optional Windows dynamic evidence -> Ghidra visualization.

ГОТОВЫЕ КОМАНДЫ ДЛЯ HTML И JSON ОТЧЕТОВ

Этот раздел можно использовать как готовую шпаргалку для терминала Visual Studio. Команды рассчитаны на PowerShell. Если Visual Studio уже открыла папку ip_full, команду cd можно пропустить.

1. Перейти в папку проекта и собрать release

```
cd "C:\Users\Templar\Desktop\вкр 10 под minGW оконченный\ip_full"
cmake --preset release
cmake --build --preset release
```

После сборки analyzer должен лежать здесь: build\release\eh_unwind_analyzer.exe. Дальше команды используют именно этот файл.

2. Один раз задать удобные переменные

```
$AN = ".\build\release\eh_unwind_analyzer.exe"
$STATIC = ".\бинарники на тестирование\бинарники\static\14_static_reuse_dominance_combo.exe"
$DYNAMIC = ".\бинарники на тестирование\бинарники\dynamic\18_dynamic_rtl_function_table.exe"
New-Item -ItemType Directory -Force -Path ".\reports\manual_static", ".\reports\manual_dynamic"
$STATIC_HTML = ".\reports\manual_static\14_static_reuse_dominance_combo.html"
$STATIC_JSON = ".\reports\manual_static\14_static_reuse_dominance_combo.json"
$DYNAMIC_HTML = ".\reports\manual_dynamic\18_dynamic_rtl_function_table.html"
$DYNAMIC_JSON = ".\reports\manual_dynamic\18_dynamic_rtl_function_table.json"
```

Переменные $AN, $STATIC, $DYNAMIC и переменные путей отчетов живут только в текущем окне терминала. Если терминал закрыт, просто введи этот блок заново.

3. Статический HTML-отчет + JSON

```
& $AN $STATIC --heuristics --strict-abi --json $STATIC_JSON --html $STATIC_HTML
Invoke-Item $STATIC_HTML
notepad $STATIC_JSON
```

Что получится: HTML откроется в браузере через Invoke-Item, а JSON откроется в Notepad. Статический отчет не запускает sample, он читает PE-файл, секции, RUNTIME_FUNCTION, UNWIND_INFO и эвристики.

4. Динамический HTML-отчет + JSON

```
& $AN $DYNAMIC --heuristics --dynamic --timeout-ms 15000 --trace-steps 64 --json $DYNAMIC_JSON --html $DYNAMIC_HTML
Invoke-Item $DYNAMIC_HTML
notepad $DYNAMIC_JSON
```

Динамический отчет запускает target под Windows Debug API. Если Windows или антивирус не даст запустить sample, статический отчет все равно можно смотреть отдельно.

5. Куда сохраняется JSON после запуска

Запуск	HTML сохраняется	JSON сохраняется
Статика	ip_full\reports\manual_static\ 14_static_reuse_dominance_combo.html	ip_full\reports\manual_static\ 14_static_reuse_dominance_combo.json
Динамика	ip_full\reports\manual_dynamic\ 18_dynamic_rtl_function_table.html	ip_full\reports\manual_dynamic\ 18_dynamic_rtl_function_table.json

Главное правило: analyzer сохраняет JSON ровно туда, что написано после параметра --json. Если указать --json .\reports\my.json, файл появится в reports\my.json. Если параметр --json не указать, JSON-файл сам по себе не создастся.

6. Быстрая проверка, что файлы появились

```
Get-Item $STATIC_HTML
Get-Item $STATIC_JSON
Get-Item $DYNAMIC_HTML
Get-Item $DYNAMIC_JSON
```

Если Get-Item пишет, что файл не найден, значит команда анализа не завершилась или путь после --html/--json был другим.

ПРИЛОЖЕНИЕ: КОМАНДЫ ДЛЯ ЗАПУСКА НА ЛЮБОЙ WINDOWS-МАШИНЕ

Этот раздел можно использовать как шпаргалку перед защитой и как инструкцию для запуска на другом Windows-ПК. Все команды ниже рассчитаны на терминал Visual Studio или обычный PowerShell. Команды специально используют переменные `$PROJECT`, `$TOOL`, `$OUT`, чтобы не переписывать длинные пути вручную.

Главная идея: один запуск анализатора всегда строит отчет по одному target-файлу. Если добавить `--dynamic`, в тот же JSON/HTML попадает и статический слой, и динамический слой конкретного запуска. Если у файла нет статических подозрений или в этом запуске не произошло runtime-событий, соответствующий блок будет присутствовать, но значения могут быть нулевыми.

1. Как открыть терминал в Visual Studio

• Открой Visual Studio и папку проекта `ip_full`, либо просто открой любую Visual Studio и перейди в терминал.

• Меню: View -> Terminal. В русской версии: Вид -> Терминал.

• Лучше выбрать PowerShell или Developer PowerShell for VS. Developer PowerShell удобен тем, что чаще уже видит компилятор MSVC.

• Все команды ниже вставляются именно в этот терминал.

2. Первая команда на любой машине

Если проект лежит не по этому пути, замени путь в кавычках на фактическую папку `ip_full`. Важно зайти именно внутрь `ip_full`, потому что там лежат `CMakeLists.txt`, `build`, `reports`, `tests` и папка с бинарниками.

```
cd "C:\Users\Templar\Desktop\вкр 10 под minGW оконченный\ip_full"

$PROJECT = (Get-Location).Path
Write-Host "Project folder:" $PROJECT
```

На другой машине путь может быть, например, `D:\Diplom\ip_full` или `C:\Users\User\Desktop\ip_full`. Меняется только первая строка `cd`, все остальные команды продолжают работать через `$PROJECT`.

3. Проверка инструментов и сборка

Если `build\release\eh_unwind_analyzer.exe` уже есть, этот шаг можно пропустить. Но на новой машине лучше сначала проверить инструменты и собрать проект заново.

```
cmake --version
ninja --version

cmake --preset release
cmake --build --preset release
```

После успешной сборки основной EXE должен появиться здесь:

```
.\build\release\eh_unwind_analyzer.exe
```

Если CMake preset не сработал, можно попробовать готовые bat-скрипты проекта. На машине с MinGW:

```
.\build_mingw.bat
```

На машине с Visual Studio/MSVC:

```
.\build_msvc.bat
```

После bat-сборки EXE может лежать не в `build\release`, а в `build\mingw-local` или `build\msvc-local`. Поэтому ниже есть универсальный блок, который сам ищет подходящий analyzer.

4. Универсальная подготовка переменных

Этот блок удобно вставлять перед любым запуском. Он создает папку для ручных отчетов, находит analyzer и задает пути к тестовым бинарникам.

```
$PROJECT = (Get-Location).Path
$OUT = Join-Path $PROJECT "reports\manual_runs"
New-Item -ItemType Directory -Force $OUT | Out-Null

$TOOL_CANDIDATES = @(
    (Join-Path $PROJECT "build\release\eh_unwind_analyzer.exe"),
    (Join-Path $PROJECT "build\mingw-local\eh_unwind_analyzer.exe"),
    (Join-Path $PROJECT "build\msvc-local\eh_unwind_analyzer.exe")
)

$TOOL = $TOOL_CANDIDATES | Where-Object { Test-Path $_ } | Select-Object -First 1
if (-not $TOOL) {
    throw "Analyzer EXE not found. Build the project first."
}

$STATIC_TARGET = Join-Path $PROJECT "бинарники на тестирование\бинарники\static\
14_static_reuse_dominance_combo.exe"
$DYNAMIC_TARGET = Join-Path $PROJECT "бинарники на тестирование\бинарники\dynamic\
18_dynamic_rtl_function_table.exe"

Write-Host "Analyzer:" $TOOL
Write-Host "Reports:" $OUT
```

Переменная	Что хранит	Зачем нужна
$PROJECT	Полный путь к папке ip_full.	Позволяет запускать команды на любой машине после одного cd.
$OUT	Папка reports\manual_runs.	Сюда складываются HTML, JSON и CSV, чтобы не искать их по всему проекту.
$TOOL	Путь к eh_unwind_analyzer.exe.	Именно эта программа анализирует target.
$STATIC_TARGET	Статический sample 14_static_reuse _dominance_combo.exe.	Хорош для демонстрации статических признаков.
$DYNAMIC_TARGET	Динамический sample 18_dynamic_rtl_function_table.exe.	Хорош для демонстрации runtime-событий Rtl*FunctionTable.

5. Статический HTML/JSON/CSV отчет

Статический запуск НЕ выполняет target-файл. Analyzer читает PE-структуру, таблицу `RUNTIME_FUNCTION`, `UNWIND_INFO`, секции, права секций, handler-адреса и строит отчет. Такой запуск безопаснее для неизвестных файлов, потому что файл не стартует как процесс.

```
$STATIC_HTML = Join-Path $OUT "static_14_combo.html"
$STATIC_JSON = Join-Path $OUT "static_14_combo.json"
$STATIC_CSV  = Join-Path $OUT "static_14_combo_functions.csv"

& $TOOL $STATIC_TARGET --heuristics --strict-abi `
    --csv $STATIC_CSV `
    --json $STATIC_JSON `
    --html $STATIC_HTML

Invoke-Item $STATIC_HTML
notepad $STATIC_JSON
```

Опция	Что делает	Что смотреть в отчете
--heuristics	Включает rules/score/findings/manual validation plan.	Карточки score, verdict, findings, manual plan.
--strict-abi	Добавляет более строгие ABI-проверки unwind metadata.	Полезно для защиты: видно, что анализ не только поверхностный.
--csv	Сохраняет таблицу функций и unwind-метаданных.	Удобно для Excel и Ghidra-подготовки.
--json	Сохраняет полный машинно-читаемый отчет.	Это главный проверяемый артефакт: все поля отчета лежат явно.
--html	Сохраняет красивый интерактивный отчет.	Открывать на защите и показывать комиссии.

Где лежат результаты этого запуска:

```
.\reports\manual_runs\static_14_combo.html
.\reports\manual_runs\static_14_combo.json
.\reports\manual_runs\static_14_combo_functions.csv
```

6. Статика для любого своего EXE/DLL

Чтобы проверить свой файл, замени `$MY_TARGET` на путь к нужному EXE или DLL. Кавычки обязательны, если в пути есть пробелы или русские буквы.

```
$MY_TARGET = "C:\Path\To\program.exe"

$MY_HTML = Join-Path $OUT "my_static.html"
$MY_JSON = Join-Path $OUT "my_static.json"
$MY_CSV  = Join-Path $OUT "my_static_functions.csv"

& $TOOL $MY_TARGET --heuristics --strict-abi `
    --csv $MY_CSV `
    --json $MY_JSON `
    --html $MY_HTML

Invoke-Item $MY_HTML
```

Если анализатор пишет, что файл не PE32+ AMD64, значит target не подходит под предмет проекта: проект рассчитан на Windows x64 PE с exception/unwind metadata.

7. Динамический HTML/JSON отчет

Динамический запуск выполняет target под Windows Debug API. Поэтому использовать его на неизвестных или вредоносных файлах нужно только в изолированной VM. Для демонстрации лучше

брать готовые безопасные samples из проекта.

```
$DYNAMIC_HTML = Join-Path $OUT "dynamic_18_rtl_function_table.html"
$DYNAMIC_JSON = Join-Path $OUT "dynamic_18_rtl_function_table.json"
$DYNAMIC_CSV  = Join-Path $OUT "dynamic_18_rtl_function_table_functions.csv"

& $TOOL $DYNAMIC_TARGET --heuristics --dynamic `
    --timeout-ms 15000 `
    --trace-steps 64 `
    --csv $DYNAMIC_CSV `
    --json $DYNAMIC_JSON `
    --html $DYNAMIC_HTML

Invoke-Item $DYNAMIC_HTML
notepad $DYNAMIC_JSON
```

Опция	Что делает	Почему важно
--dynamic	Запускает target под отладчиком и собирает runtime evidence.	В отчете появляется блок dynamic_validation.
--timeout-ms 15000	Ограничивает запуск 15 секундами.	Не дает зависшему target держать анализ бесконечно.
--trace-steps 64	Сохраняет ограниченный RIP trace после handler hit.	Помогает показать, куда пошло выполнение.
--json/--html	Сохраняет один отчет с static + dynamic данными.	JSON для проверки, HTML для просмотра человеком.

В динамическом отчете смотреть: `dynamic_validation.enabled`, `ran`, `ok`, `events`, `exceptions_seen`, `veh_registrations`, `vch_registrations`, `uef_set_calls`, `rtl_add_function_table_calls`, `rtl_install_function_table_cb_calls`, `trace`.

8. Четыре готовых dynamic-сценария

В проекте есть четыре показательных dynamic sample. Команда одна и та же, меняется только `$DYNAMIC_TARGET` и имя выходного файла.

Файл	Что демонстрирует	Какие поля смотреть
15_dynamic_veh.exe	Регистрация и срабатывание VEH.	veh_registrations, veh_hits, events.
16_dynamic_vch.exe	Регистрация и срабатывание VCH.	vch_registrations, vch_hits, events.
17_dynamic_uef.exe	SetUnhandledExceptionFilter и UEF flow.	uef_set_calls, uef_hits, exceptions_seen.
18_dynamic_rtl_function_table.exe	RtlAddFunctionTable и RtlInstallFunctionTableCallback.	rtl_add_function_table_calls, rtl_install_function_table_cb_calls.

```
$DYNAMIC_TARGET = Join-Path $PROJECT "бинарники на тестирование\бинарники\dynamic\
15_dynamic_veh.exe"
$DYNAMIC_HTML = Join-Path $OUT "dynamic_15_veh.html"
$DYNAMIC_JSON = Join-Path $OUT "dynamic_15_veh.json"

& $TOOL $DYNAMIC_TARGET --heuristics --dynamic --timeout-ms 15000 --trace-steps 64 `
    --json $DYNAMIC_JSON --html $DYNAMIC_HTML

Invoke-Item $DYNAMIC_HTML
```

9. Смешанный отчет: статика + динамика в одном HTML

Смешанный отчет создается не отдельной командой `merge`, а обычным запуском с `--heuristics --dynamic`. Analyzer сначала читает файл как PE и строит статические признаки, а затем запускает target и добавляет динамические события в тот же JSON/HTML.

```
$MIXED_TARGET = Join-Path $PROJECT "бинарники на тестирование\бинарники\dynamic\
18_dynamic_rtl_function_table.exe"

$MIXED_HTML = Join-Path $OUT "mixed_static_dynamic.html"
$MIXED_JSON = Join-Path $OUT "mixed_static_dynamic.json"
$MIXED_CSV  = Join-Path $OUT "mixed_static_dynamic_functions.csv"

& $TOOL $MIXED_TARGET --heuristics --dynamic `
    --timeout-ms 15000 `
    --trace-steps 64 `
    --csv $MIXED_CSV `
    --json $MIXED_JSON `
    --html $MIXED_HTML

Invoke-Item $MIXED_HTML
notepad $MIXED_JSON
```

Что получится: один HTML `mixed_static_dynamic.html`. В нем будут верхние карточки, PE/sections, static findings, final assessment, manual validation plan и dynamic validation. Если у выбранного target нет статических findings, они будут равны нулю, но статический слой все равно был выполнен.

10. Смешанный отчет на sample со статическими аномалиями

Если нужно показать сильную статику и одновременно включенный dynamic-блок, можно запустить `--dynamic` на статическом combo sample. Важно: dynamic-блок может показать нули по runtime-событиям, потому что конкретный sample может не регистрировать VEH/VCH/UEF/Rtl* во время запуска. Это нормально: отчет показывает факт, а не дорисовывает события.

```
$MIXED_TARGET = Join-Path $PROJECT "бинарники на тестирование\бинарники\static\
14_static_reuse_dominance_combo.exe"

$MIXED_HTML = Join-Path $OUT "mixed_14_static_combo.html"
$MIXED_JSON = Join-Path $OUT "mixed_14_static_combo.json"
$MIXED_CSV  = Join-Path $OUT "mixed_14_static_combo_functions.csv"

& $TOOL $MIXED_TARGET --heuristics --strict-abi --dynamic `
    --timeout-ms 15000 `
    --trace-steps 64 `
    --csv $MIXED_CSV `
    --json $MIXED_JSON `
    --html $MIXED_HTML

Invoke-Item $MIXED_HTML
```

Как объяснять комиссии: `--dynamic` не означает, что в каждом файле обязательно найдутся runtime-события. Он означает, что анализатор попытался выполнить target и записал наблюдения. Если событий нет, это тоже результат конкретного эксперимента.

11. Если есть внешний combo-бинарник

Если у тебя есть отдельный файл, который одновременно содержит статические аномалии и runtime-поведение, например combo sample из прошлой папки, запускай его той же командой. Меняется только путь `$MIXED_TARGET`.

```
$MIXED_TARGET = "C:\Path\To\28_combo_rtl_xdata_renamed_writable.exe"

$MIXED_HTML = Join-Path $OUT "mixed_external_combo.html"
$MIXED_JSON = Join-Path $OUT "mixed_external_combo.json"
$MIXED_CSV  = Join-Path $OUT "mixed_external_combo_functions.csv"
```

```
& $TOOL $MIXED_TARGET --heuristics --strict-abi --dynamic `
    --timeout-ms 15000 `
    --trace-steps 64 `
    --csv $MIXED_CSV `
    --json $MIXED_JSON `
    --html $MIXED_HTML

Invoke-Item $MIXED_HTML
```

Тогда в одном HTML можно получить и static findings, и dynamic_validation events. Но это зависит от самого бинарника: analyzer не склеивает результаты разных файлов в один отчет, потому что так потерялась бы проверяемость SHA-256, пути target и доказательная связность.

12. Команды для Ghidra CSV

Для Ghidra нужен отдельный CSV с диапазонами функций и комментариями. Его можно делать вместе с обычным отчетом или отдельной командой.

```
$GHIDRA_CSV = Join-Path $OUT "ghidra_unwind.csv"

& $TOOL $STATIC_TARGET --heuristics --ghidra-csv $GHIDRA_CSV

Write-Host "Ghidra CSV:" $GHIDRA_CSV
```

Дальше в Ghidra: File -> Import File -> выбрать тот же EXE -> Analyze Now. Потом Window -> Script Manager -> подключить папку `ghidra` из проекта -> запустить `import_unwind_csv.py` -> выбрать созданный `ghidra_unwind.csv`.

13. Быстрая демонстрация перед защитой

Если нужно быстро показать проект за несколько минут, используй этот порядок. Он дает один спокойный отчет, один статически подозрительный отчет и один dynamic-отчет.

```
cd "C:\Users\Templar\Desktop\вкр 10 под minGW оконченный\ip_full"

$PROJECT = (Get-Location).Path
$OUT = Join-Path $PROJECT "reports\defense_demo"
New-Item -ItemType Directory -Force $OUT | Out-Null
$TOOL = Join-Path $PROJECT "build\release\eh_unwind_analyzer.exe"

$BENIGN = Join-Path $PROJECT "бинарники на тестирование\бинарники\static\01_benign_minimal.
exe"
$STATIC = Join-Path $PROJECT "бинарники на тестирование\бинарники\static\
14_static_reuse_dominance_combo.exe"
$DYNAMIC = Join-Path $PROJECT "бинарники на тестирование\бинарники\dynamic\
18_dynamic_rtl_function_table.exe"

& $TOOL $BENIGN --heuristics --json (Join-Path $OUT "01_benign.json") --html (Join-Path $OUT
"01_benign.html")
& $TOOL $STATIC --heuristics --strict-abi --json (Join-Path $OUT "14_static_combo.json") --
html (Join-Path $OUT "14_static_combo.html")
& $TOOL $DYNAMIC --heuristics --dynamic --timeout-ms 15000 --trace-steps 64 --json (Join-
Path $OUT "18_dynamic_rtl.json") --html (Join-Path $OUT "18_dynamic_rtl.html")

Invoke-Item (Join-Path $OUT "01_benign.html")
Invoke-Item (Join-Path $OUT "14_static_combo.html")
Invoke-Item (Join-Path $OUT "18_dynamic_rtl.html")
```

Такой набор удобен для защиты: benign показывает, что analyzer не всегда ругается; static combo показывает сильные структурные признаки; dynamic sample показывает runtime collector и события Windows Debug API.

14. Где сохраняется JSON и почему это важно

JSON сохраняется ровно туда, куда указывает параметр `--json`. Если параметр `--json` не указан, JSON-файл не создается. HTML аналогично создается только при наличии `--html`, CSV - только при наличии `--csv`.

```
Get-Item (Join-Path $OUT "static_14_combo.html")
Get-Item (Join-Path $OUT "static_14_combo.json")
Get-Item (Join-Path $OUT "dynamic_18_rtl_function_table.html")
Get-Item (Join-Path $OUT "dynamic_18_rtl_function_table.json")
```

На защите можно открыть HTML для красивого просмотра, а JSON держать рядом как доказательство, что поля отчета не нарисованы вручную: там есть SHA-256, версия analyzer, команда запуска, counters, findings и dynamic_validation.

15. Частые ошибки

Ошибка	Причина	Что сделать
Analyzer EXE not found	Проект не собран или EXE лежит в другой build-папке.	Выполнить `cmake --preset release` и `cmake --build --preset release`, либо `build_mingw.bat`/`build_msvc.bat`.
cmake не найден	CMake не установлен или не добавлен в PATH.	Открыть Developer PowerShell, установить CMake или добавить его в PATH.
ninja не найден	Preset использует Ninja generator.	Установить Ninja или открыть среду, где он есть.

Ошибка	Причина	Что сделать
HTML не открылся	Файл не создан или браузер не связан с HTML.	Проверить `Get-Item $HTML`; открыть файл вручную из reports.
Dynamic failed	Target завершился необычно, не x64, нет прав debug или сработал timeout.	Использовать samples проекта, увеличить timeout, запускать в VM/от имени пользователя с правами.
В dynamic блоке нули	Конкретный запуск не вызвал наблюдаемые runtime-события.	Это не ошибка: проверить другой dynamic sample или объяснить ограничение конкретного запуска.
Статические findings ноль	В PE не найдено сильных структурных признаков.	Для демонстрации взять `14_static_reuse_dominance_combo.exe` или другой suspicious sample.

16. Короткая формула для объяснения комиссии

Статический отчет отвечает на вопрос: что записано в PE-файле и насколько это похоже на нормальную unwind/EH-структуру. Динамический отчет отвечает на вопрос: что реально произошло при конкретном запуске под Windows Debug API. Смешанный отчет - это один HTML/JSON по одному target-файлу, где оба слоя лежат рядом и могут либо подтверждать друг друга, либо честно показывать, что один из слоев в данном эксперименте не дал события.

17. ОТДЕЛЬНЫЕ КОМАНДЫ ДЛЯ СМЕШАННОГО COMBO-БИНАРНИКА

Этот пункт нужен именно для случая, когда один и тот же target-файл должен дать оба слоя: статические аномалии в PE/unwind metadata и динамические события во время запуска. Это не склейка двух разных отчетов. Это один запуск analyzer по одному EXE, поэтому SHA-256, путь к файлу и доказательная цепочка остаются честными.

Если такого mixed/combo EXE в текущей папке нет, команда все равно правильная: надо только подставить путь к тому файлу, где реально есть и статическое вмешательство, и runtime-поведение.

17.1. Найти mixed/combo EXE

Сначала можно поискать подходящие файлы в проекте. Если файл лежит в другой папке, например в старой папке `ip_full/reports/combo` или на рабочем столе, путь надо указать вручную.

```
cd "C:\Users\Templar\Desktop\вкр 10 под minGW оконченный\ip_full"

$PROJECT = (Get-Location).Path

Get-ChildItem -Path $PROJECT -Recurse -Filter "*combo*.exe" -ErrorAction SilentlyContinue
Get-ChildItem -Path $PROJECT -Recurse -Filter "*mixed*.exe" -ErrorAction SilentlyContinue
Get-ChildItem -Path $PROJECT -Recurse -Filter "*rtl*xdata*.exe" -ErrorAction SilentlyContinue
```

17.2. Запустить mixed/combo отчет в HTML, JSON и CSV

В строке `$MIXED_TARGET` укажи реальный путь к смешанному бинарнику. Пример ниже написан для файла `28_combo_rtl_xdata_renamed_writable.exe`, потому что такой тип sample как раз удобен для демонстрации: static layer видит необычное/изменяемое размещение unwind metadata, a dynamic layer может увидеть Rtl*FunctionTable события.

```
$PROJECT = (Get-Location).Path
$TOOL = Join-Path $PROJECT "build\release\eh_unwind_analyzer.exe"
$OUT = Join-Path $PROJECT "reports\mixed_combo_manual"
New-Item -ItemType Directory -Force $OUT | Out-Null

$MIXED_TARGET = "C:\Path\To\28_combo_rtl_xdata_renamed_writable.exe"

$MIXED_HTML = Join-Path $OUT "28_combo_static_dynamic.html"
$MIXED_JSON = Join-Path $OUT "28_combo_static_dynamic.json"
$MIXED_CSV  = Join-Path $OUT "28_combo_static_dynamic_functions.csv"

& $TOOL $MIXED_TARGET --heuristics --strict-abi --dynamic `
    --timeout-ms 15000 `
    --trace-steps 64 `
    --csv $MIXED_CSV `
    --json $MIXED_JSON `
    --html $MIXED_HTML

Invoke-Item $MIXED_HTML
notepad $MIXED_JSON
```

17.3. Если analyzer лежит не в build\release

На другой машине проект может быть собран через MinGW или MSVC script, тогда EXE может лежать в другой build-папке. Этот вариант сам найдет analyzer.

```
$TOOL_CANDIDATES = @(
    (Join-Path $PROJECT "build\release\eh_unwind_analyzer.exe"),
    (Join-Path $PROJECT "build\mingw-local\eh_unwind_analyzer.exe"),
    (Join-Path $PROJECT "build\msvc-local\eh_unwind_analyzer.exe")
)

$TOOL = $TOOL_CANDIDATES | Where-Object { Test-Path $_ } | Select-Object -First 1
if (-not $TOOL) {
```

```
    throw "Analyzer EXE not found. Build the project first."
}
```

17.4. Проверить, что отчет реально смешанный

После запуска можно проверить JSON прямо из PowerShell. Для настоящего mixed/combo case желательно увидеть: `Static findings count` больше нуля и `Dynamic events count` больше нуля.

```
$j = Get-Content $MIXED_JSON -Raw | ConvertFrom-Json

"Static score: " + $j.heuristic_analysis.score
"Static findings count: " + @($j.heuristic_analysis.findings).Count
"Dynamic enabled: " + $j.dynamic_validation.enabled
"Dynamic events count: " + @($j.dynamic_validation.events).Count
"RtlAddFunctionTable calls: " + $j.dynamic_validation.rtl_add_function_table_calls
"RtlInstallFunctionTableCallback calls: " + $j.dynamic_validation.
rtl_install_function_table_cb_calls
```

Что получилось	Что означает	Как объяснить
Static findings > 0 и Dynamic events > 0	Это настоящий смешанный отчет: один файл дал и static evidence, и runtime evidence.	Показывать комиссии как главный combined case.
Static findings > 0, Dynamic events = 0	Статика нашла нарушения, но конкретный запуск не дошел до наблюдаемого runtime-поведения.	Это не ошибка: dynamic layer ограничен конкретным путем выполнения.
Static findings = 0, Dynamic events > 0	Файл в структуре PE выглядит спокойно, но при запуске регистрирует runtime handlers/tables.	Показывать как пример пользы dynamic validation.
Оба ноль	Этот target не является показательным для демонстрации аномалий.	Взять другой sample или проверить, что путь `$MIXED_TARGET` указан правильно.

17.5. Где смотреть результат

```
.\reports\mixed_combo_manual\28_combo_static_dynamic.html
.\reports\mixed_combo_manual\28_combo_static_dynamic.json
.\reports\mixed_combo_manual\28_combo_static_dynamic_functions.csv
```

HTML открывается для просмотра человеком. JSON нужен для строгой проверки полей и счетчиков. CSV нужен для таблицы функций и дальнейшего импорта/анализа.

17.6. Короткая формулировка для защиты

Смешанный отчет означает: один и тот же бинарник сначала разобран статически как PE/unwind metadata, а затем выполнен под Windows Debug API. Поэтому в одном HTML видны и структурные признаки из файла, и события конкретного запуска. Если оба слоя ненулевые, вывод сильнее, потому что static evidence и runtime evidence поддерживают друг друга.